GEEK ROOM GLOBAL AI HACKATHON · BIFROST × TAVILY

FreshGuard

Freshness checks for LLM answers, at the AI gateway. Outdated claims caught, re-checked on the live web, and reported with sources.

Sidharth C S

Problem statement #4 (Bifrost) + #5 (Tavily)

THE PROBLEM

Models state outdated facts with full confidence

ASKED IN NOVEMBER 2026

“Who is the current Prime Minister of the United Kingdom?”

Gemini: **“The current Prime Minister is Keir Starmer.”**

WHAT THE LIVE WEB SAYS

Andy Burnham has been Prime Minister since 20 July 2026

Source: House of Lords Library, found by FreshGuard

Same for prices, API versions, laws, and anything “latest”. Users can’t tell which sentences are stale.

WHY THE GATEWAY

The gateway sees every answer first

01

Training data ages every day

Versions, prices, leaders and laws change after the cutoff.

02

Every app fixes it alone

Per-app retrieval or fact-checking, built again for each product.

03

The gateway already sits in the path

Bifrost routes every model call, so one plugin covers every app.

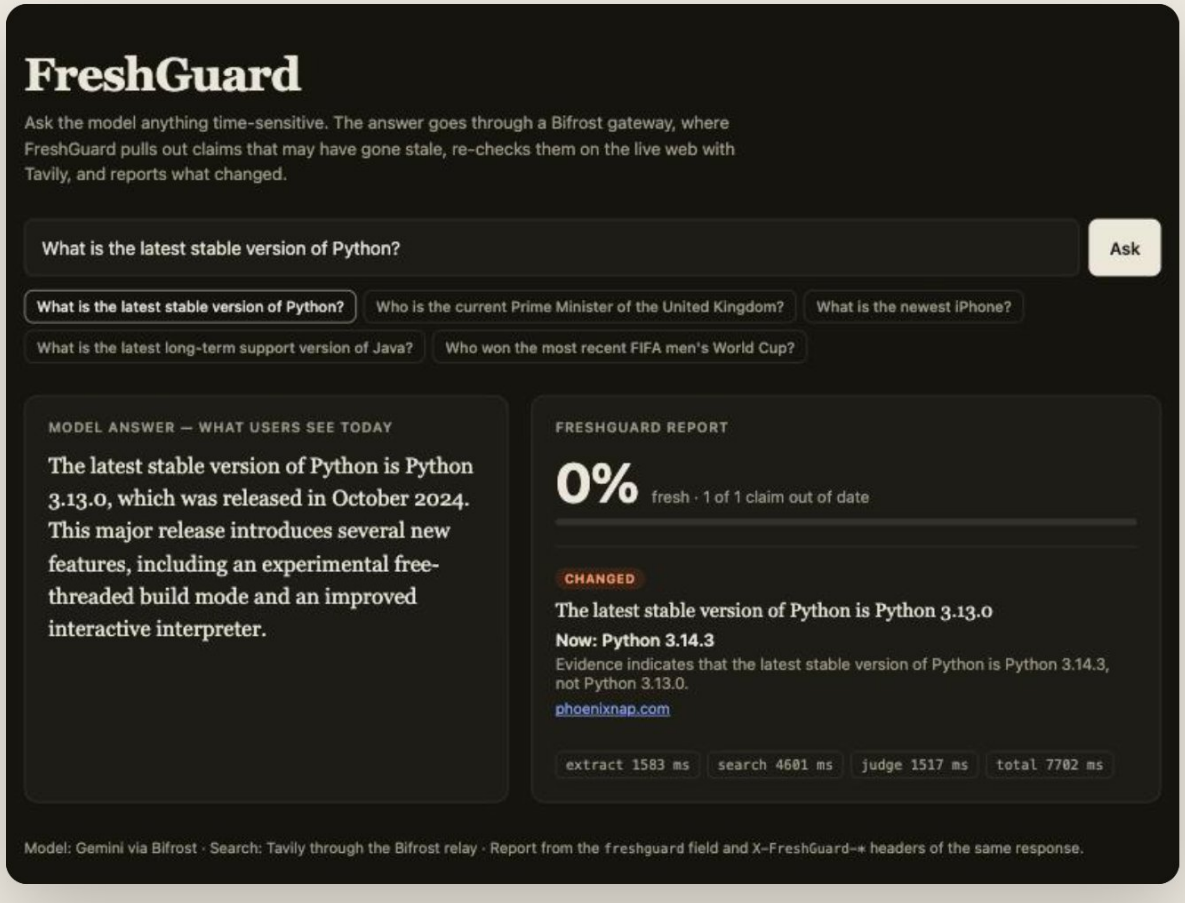
THE SOLUTION

FreshGuard, a Bifrost plugin

Every checked answer comes back with a verdict per claim, the current value, and the sources it rests on.

```
X-FreshGuard-Score: 0.00
X-FreshGuard-Stale: 1
"freshguard": { claims: [ … ] }
```

No change to the calling app.



ARCHITECTURE

Four steps after the answer is ready

1 · EXTRACT

Up to 5 time-sensitive claims, each with a query and its official site

Model call through Bifrost

→

2 · SEARCH

Open web plus the official site, last 12 months, in parallel

Tavily through the Bifrost relay

→

3 · JUDGE

Supported, changed or unverifiable, from evidence only

Model call through Bifrost

→

4 · ATTACH

Score headers and a freshguard block in the JSON

Same response, no app change

Go plugin · PostHook + StreamChunkHook · streaming or not · verified claims cached 1 h · opt-in per virtual key

BOTH TECHNOLOGIES

Bifrost does the plumbing, Tavily does the knowing

Bifrost

- Hosts the plugin in the request path
- Runs the model calls, with fallbacks
- Holds the Tavily key in a relay, not in the plugin
- Bills every check to the caller’s virtual key

Tavily

- Live web search per claim
- Recency filter: results from the last year
- Readable page snippets the judge can cite
- The only evidence a verdict may rest on

```
POST /relay/tavily/search · x-bf-vk: <caller’s key>
```

Bifrost adds the Tavily credential, checks the key’s relay grant, and records the spend.

DESIGN CHOICES

Built to sit in production traffic

Fails open

A failed search or bad model reply never breaks the user’s answer.

Evidence only

Cited URLs not in the search results are dropped: no invented sources.

No recursion

Its own model calls carry a marker header and are never checked.

No key leaks

A virtual key is forwarded only to this gateway, never to a third party.

Operator control

On for chosen virtual keys, for all traffic, or per request by header.

Tested

Unit tests for the pipeline, eligibility rules and key forwarding.

RESULTS · 37 HAND-LABELLED ANSWERS

Every stale claim caught, nothing true flagged

18/18
outdated claims caught (target ≥ 70%)

0/19
current or timeless claims wrongly flagged (target ≤ 15%)

5/5
stale prices, wages and laws caught, with the current value

2.5 s
one uncontended check (target < 3 s)

Gemini 3.5 Flash-Lite, Tavily basic search via the Bifrost relay. 18 stale, 12 current, 7 timeless. One check hit a model timeout and passed on re-run. Full tables in eval/.

WHAT THE EVAL FIXED

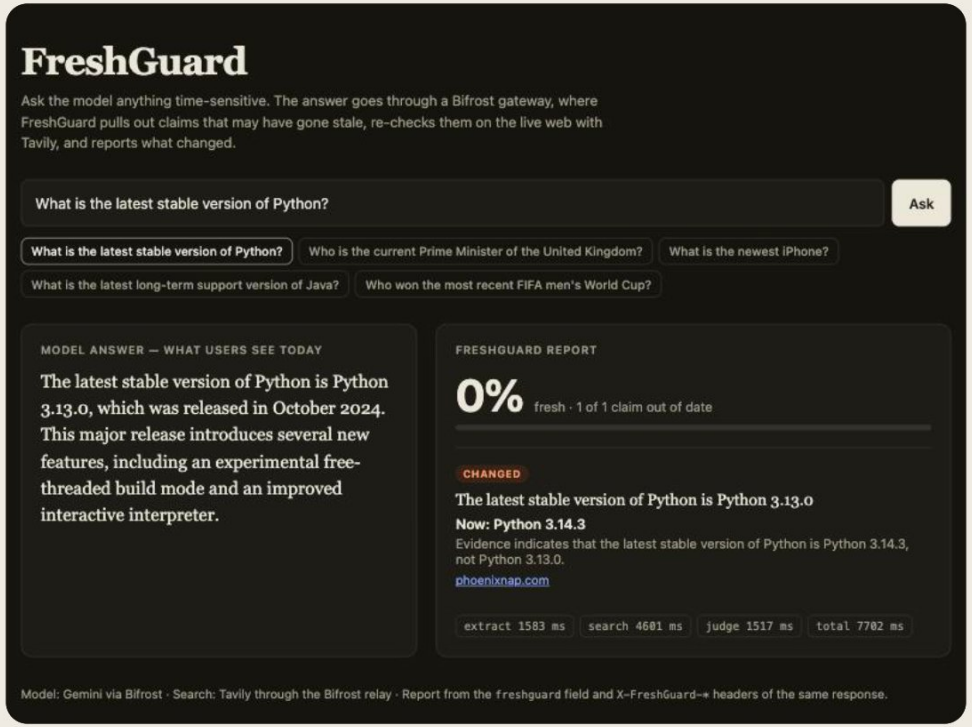
First run: 77% caught, 1 wrong, 3 errors

What went wrong	Fix
Gemini free-tier rate limits and one timeout	Bifrost provider retries with backoff
A raw line break broke the model's JSON	Model calls request JSON mode
"Starmer became PM in 2024" rewritten as "is PM"	Claims keep their tense; dated past events skipped
2023 blog posts used as evidence	Year in the query; Tavily limited to the last year

After the fixes: 13/13 caught, 0/17 wrong, 0 errors, at both search depths.

WORKING PROTOTYPE

Live demo: answer on the left, report on the right





Ask anything time-sensitive, or pick a preset.

Each claim shows its verdict, current value, sources and per-step timing.

The demo server keeps the virtual key server-side.

```
./dev.sh run · python3 demo/serve.py · http://localhost:8000
```

LIMITS AND NEXT STEPS

What is left

Known limits

- About 2.5 s per new claim; repeats come from the cache
- Latency follows the model provider
- Stream reports ride in raw_response
- Needs the dynamic build; Docker does it

Next

- Upstream proposal to maximhq/bifrost
- A plugin-metadata field on chunks
- Cache shared across gateway replicas
- A larger, shared eval set

Every verdict ships with its sources, so any call FreshGuard makes can be checked.

RUN IT

FreshGuard

Stale answers caught at the gateway, with sources.

```
cp .env.example .env
docker compose up --build
open http://localhost:8000
```

Live: freshguard-demo.onrender.com Code: github.com/sidharth-cs/freshguard Sidharth C S · Apache-2.0